

PRACTICE EXERCISES OF THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

Student's name: Do Quoc Huy

Student's code: 23520604

PRACTICE EXERCISE #3: USING INTERRUPT

I. Student preparation

- Knowledge of how to install and use Interrupt.
- Trong vi điều khiển 8051, **ngắt** (interrupt) là cơ chế tạm dừng chương trình chính để xử lý một sự kiện khẩn cấp, sau đó quay lại tiếp tục chương trình chính.
- Trong 8051 có 6 nguồn ngắt : 1 ngắt reset có mức ưu tiên cao nhất, 2 nguồn ngắt ngoài, 2 nguồn ngắt từ 2 timer và 1 nguồn ngắt từ cổng nối tiếp. Khi hệ thống reset thì tất cả các nguồn ngắt đều bị cấm (trừ ngắt reset), để cho phép các ngắt hoạt động cần lập trình nạp các giá trị phù hợp vào thanh ghi IE.

Ngắt	Cờ ngắt	Địa chỉ trình phục vụ ngắt	Số thứ tự ngắt
Reset	-	0000h	-
Ngắt ngoài 0	IE0	0003h	0
Timer 0	TF0	000Bh	1
Ngắt ngoài 1	IE1	0013h	2
Timer 1	TF1	001Bh	3
Ngắt truyền thông	RI/TI	0023h	4

Bảng vector ngắt

Bảng 6-1 Các bit chức năng của thanh ghi IE

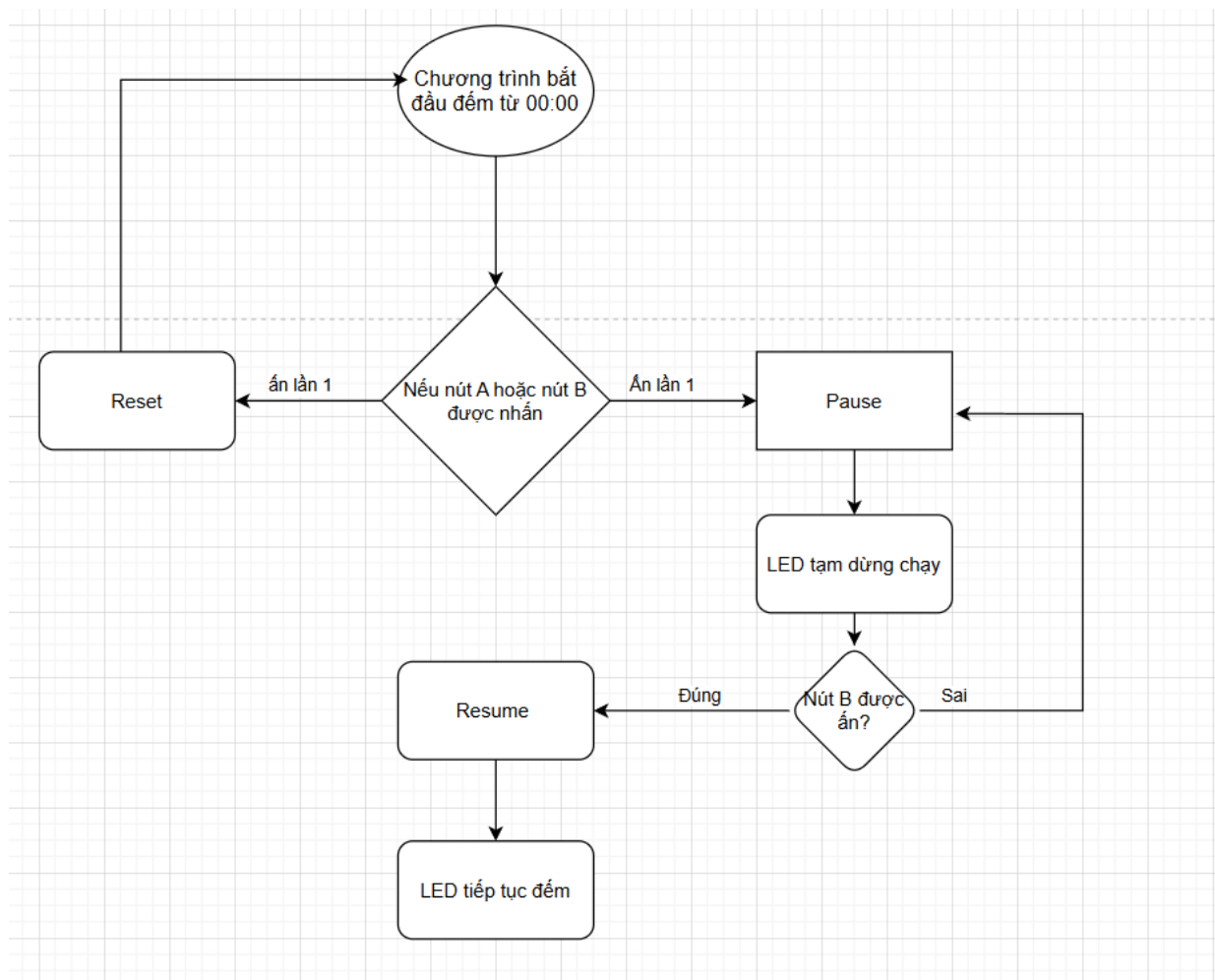
Bit	Ký hiệu	Địa chỉ	Chức năng (1: cho phép, 0: cấm)
IE7	EA	AFH	Cho phép/cấm toàn bộ các ngắt
IE6	-	AEH	Không sử dụng
IE5	ET2	ADH	Cho phép ngắt Timer 2 (8052)
IE4	ES	ACH	Cho phép ngắt cổng nối tiếp
IE3	ET1	ABH	Cho phép ngắt Timer 1
IE2	EX1	AAH	Cho phép ngắt ngoài từ INT1
IE1	ET0	A9H	Cho phép ngắt Timer 0
IE0	EX0	A8H	Cho phép ngắt ngoài từ INT0

II. Practice content

1. Present and draw a flowchart to handle 2 buttons with the following functions:

- Button A: Pause/Resume stopwatch
- Button B: Reset the stopwatch.

- Lưu đồ thuật toán :



- Sơ đồ kết nối:

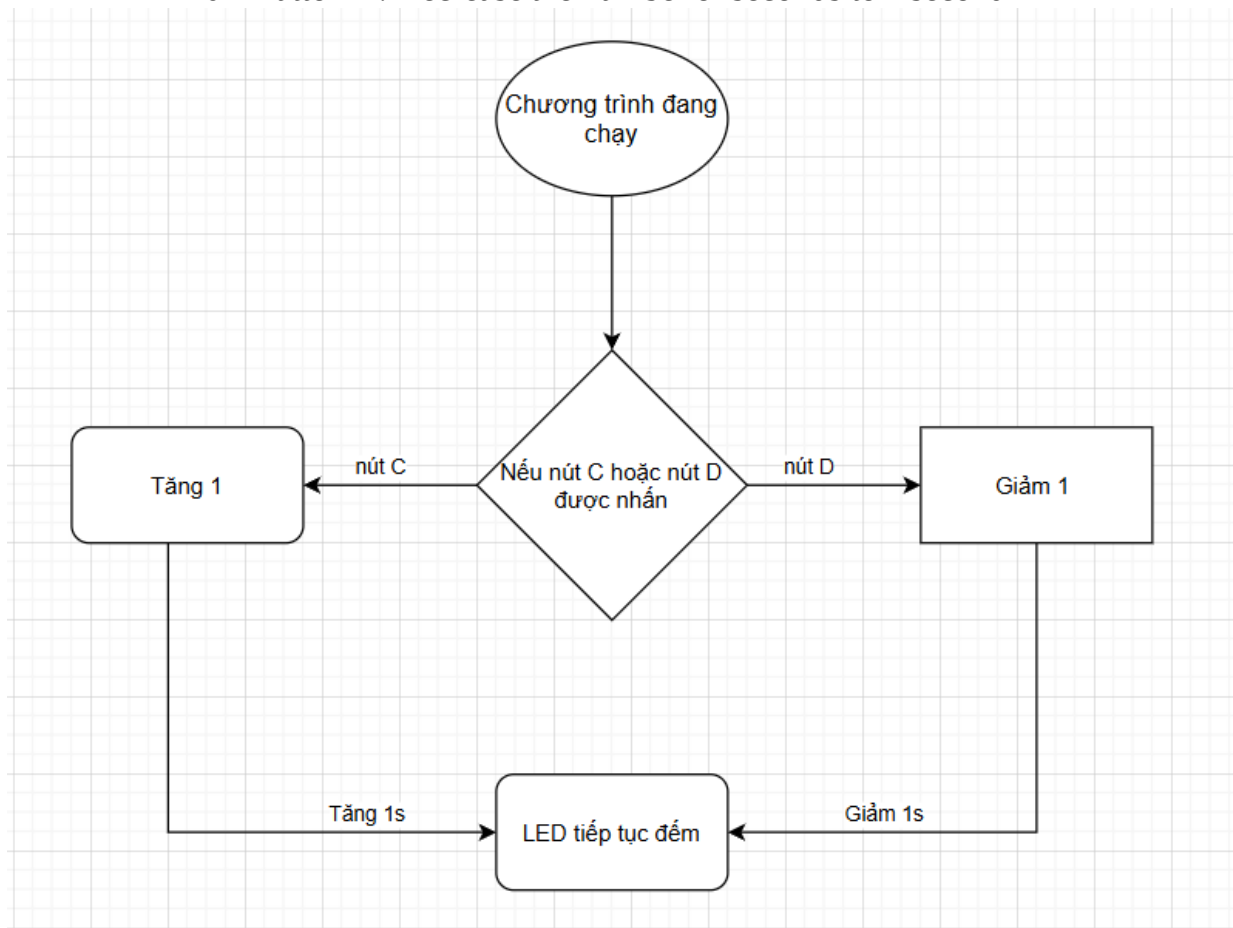
- Vi điều khiển AT89C51: Port P2.0-P2.3 dùng nối với IC Decoder 74247 để điều khiển LED 7 thanh, Cổng số hiển thị số 2 kết hợp với ngõ logic NOT dùng để nối với chân hiển thị dấu chấm của bảng LED 7-Segment. Port 3.2 và Port 3.3 (Ngắt 0&1) được nối lần lượt vào nút Pause/Resume và nút Reset.
- IC Decoder 74247 nối với bảng hiển thị LED 7-Segment để mã hóa tín hiệu đầu vào từ các chân P2.0-P2.3
- 2 Button nối vào 2 Ngắt với chức năng Pause/Resume và nút Reset
- Bảng hiển thị LED 7-Segment

2. Using AT89C51/AT89C52 in combination with 4 7-Segment LED modules and 2 buttons above, design a Sport clock circuit with the ability to count accurately to 1% of seconds, counting range from 00.00 seconds to 99.99 seconds and has 2 buttons to control Pause/Resume and Reset.

III. Exercises

Add 2 buttons to the Sport watch with the following function:

- Button C: Increase the number of seconds counting to 1 second
- Button D: Decrease the number of seconds to 1 second



- Chân P0.0 và chân P0.1 được nối lần lượt vào nút Tăng 1 và nút Giảm 1 với hai chức năng cơ bản là tăng 1 giây và giảm 1 giây

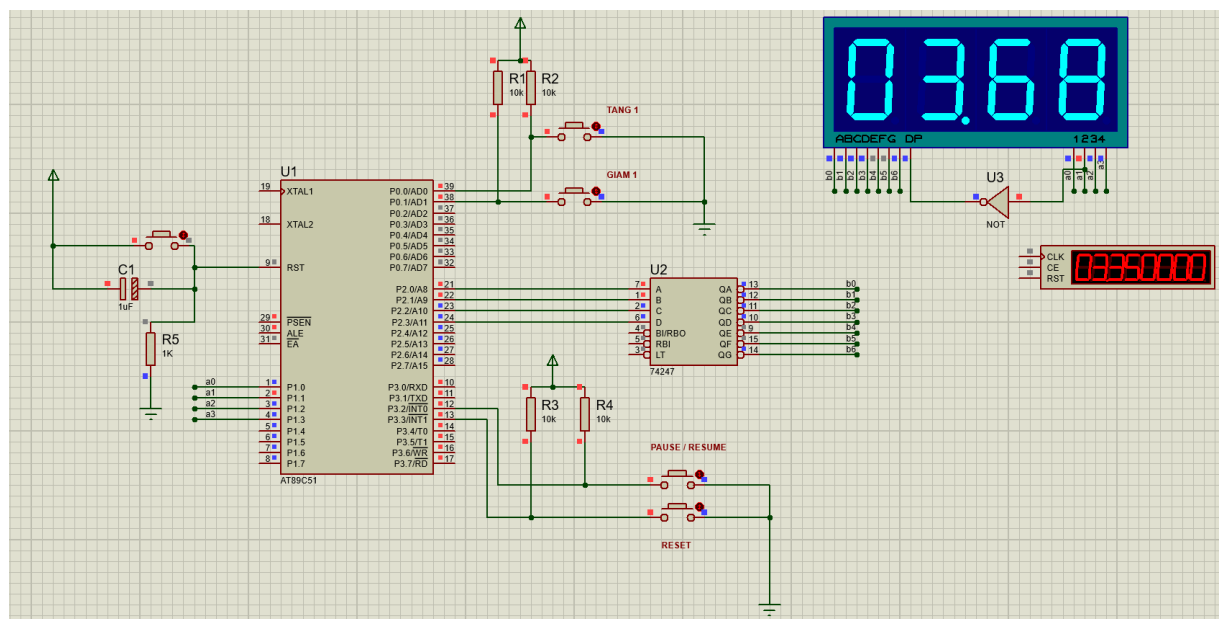
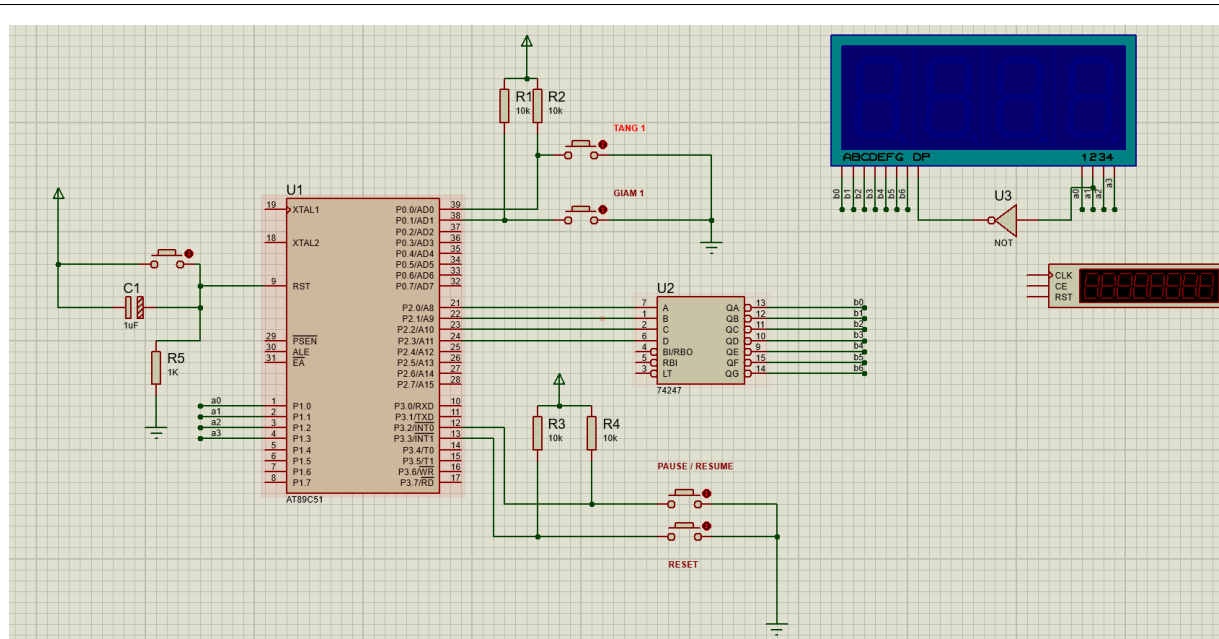
IV. Report

Compress design files and report files into a file named as follows:

[<LAB...>]-[<Student code>]-Full name

The required report file contains the following contents:

1. Design result (screenshot and pasted in the report).



2. Explain the operating principle of the effects, accompanied by a video (send a Google Drive link) to demonstrate the circuit operation in case the instructor cannot run the design file.

- **Link google drive:**

<https://drive.google.com/drive/folders/1yPwSQZeIUtRiw2JIjmNTOCfDr1tx7EyO?usp=sharing>

- **Giải thích:**

Khởi tạo: <pre> \$NOMOD51 \$INCLUDE (8051.MCU) P0_0_LAST BIT 20H.0 P0_1_LAST BIT 20H.1 ORG 0000H JMP START ORG 00BH JMP NGAT_TIMER ORG 0003H JMP INT0_PAUSE_RESUME ORG 0013H LJMP INT1_RESET ORG 0030H MILISECOND EQU R0 SECOND EQU R1 </pre>	- Định nghĩa 2 cờ trạng thái (bit) trong RAM nội. - Dùng để ghi nhớ trạng thái trước đó của các nút P0.0 và P0.1 - Khởi tạo Timer và Interrupt theo bảng vector ngắt - Từ địa chỉ 0030H là nơi bắt đầu chương trình chính - Đặt R0 là thanh ghi lưu biến mili giây - Đặt R1 là thanh ghi lưu biến giây
Các hàm delay: <pre> DELAY: MOV TMOD, #01H MOV TH0, #0DCH MOV TL0, #00H CLR TF0 SETB TR0 JNB TF0, \$ RET DELAY_100MS: MOV R5, #100 LOOP1: ACALL DELAY DJNZ R5, LOOP1 RET </pre>	- Khai báo Timer 0 mode 1 (16 bit) - Timer 0 mode 1 sẽ đếm từ giá trị 0xDC00 đến 0xFFFF $\Rightarrow$ tổng cộng là $0xFFFF - 0xDC00 + 1 = 0x2400 = 9216$ lần đếm. - Thời gian timer sẽ là : $9216 \times 1\mu s = 9.216 \text{ ms} \sim 10\text{ms}$

<pre> ORG 0100H START: MOV IE, #87H SETB IT0 SETB IT1 SETB P0_0_LAST SETB P0_1_LAST MOV SECOND, #0 MOV MILLISECOND, #0 LOOP: LCALL DISPLAY JNB P0.0, WHEN_P0_0 SETB P0_0_LAST JMP WHEN_P0_1 </pre>	- Cho phép ngắt toàn cục: bật các bit: EA, ET0, EX0, EX1 #87H = 10000111B => EA=1, ET0=1, EX1=1, EX0=1 - Cấu hình INT0, INT1 hoạt động ở chế độ cạnh xuống - Đặt cờ P0_0_LAST = 1, P0_1_LAST = 1 để lưu trạng thái trước đó và phát hiện cạnh xuống - Khởi tạo các biến đếm giây và mili giây - Hiển thị thời gian được đếm và liên tục kiểm tra trạng thái nút nhấn P0.0 và P0.1
Hàm hiển thị: <pre> DISPLAY: MOV A, MILLISECOND MOV B, #10 DIV AB MOV 010H, A MOV 011H, B MOV P1, #00000000B SETB P1.3 MOV P2, 011H ACALL DELAY CLR P1.3 SETB P1.2 MOV P2, 010H ACALL DELAY CLR P1.2 </pre>	- DIV AB dùng để tách số hàng chục và số hàng đơn vị - Sử dụng các ô nhớ để lưu tạm các giá trị giây, mili giây. - Quy trình hiển thị 1 LED: bật LED → gán P2 bởi giá trị các ô nhớ tạm → P2 truyền qua ic 74247 và truyền đến LED → tắt LED hiện tại và chuyển đến LED tiếp theo. - Khi có sự chuyển tiếp của các LED nhanh khiến mắt thường không thể nhìn thấy được → Thủ thuật quét LED

<pre> MOV A, SECOND MOV B, #10 DIV AB MOV 012H, A MOV 014H, B MOV P1, #00000000B SETB P1.1 MOV P2, 014H ACALL DELAY CLR P1.1 SETB P1.0 MOV P2, 012H ACALL DELAY CLR P1.0 RET </pre>	
Hàm nhấn nút P0.0 <pre> WHEN_P0_0: JB P0_0_LAST, P0_0_DOWN JMP WHEN_P0_1 P0_0_DOWN: CLR P0_0_LAST CALL NGAT_TANG1 </pre>	- Nếu nút tại P0.0 được nhấn (P0_0_LAST = 1) thì nhảy đến P0_0_DOWN - Ngược lại kiểm tra nút tại P0.1 - Khi nút được nhấn ta gọi hàm NGAT_TANG1 và cập nhật lại cờ P0_0_LAST
Hàm nhấn nút P0.1	- Nếu nút tại P0.1 được nhấn thì nhảy WHEN_P0_1_WAS_1 - Nếu nút vừa được nhấn (P0_1_LAST = 1) nhảy đến P0_1_DOWN - Ta thực hiện hàm NGAT_GIAM1 và cập nhật lại cờ P0_1_LAST - Ngược lại nếu nút chưa được nhấn, ta ghi nhận trạng thái (lúc này đang nhả nút)

<pre> WHEN_P0_1: JNB P0.1, WHEN_P0_1_WAS_1 SETB P0_1_LAST JMP SKIP WHEN_P0_1_WAS_1: JB P0_1_LAST, P0_1_DOWN JMP SKIP P0_1_DOWN: CLR P0_1_LAST CALL NGAT_GIAM1 SKIP: JMP LOOP </pre>	
Hàm nút nhấn INT0 VÀ INT1 <pre> INT0_PAUSE_RESUME: CPL TR0 RETI INT1_RESET: MOV SECOND, #0 MOV MILISECOND, #0 RETI </pre>	- Khi nhấn nút ta đảo bit TR0 – bật/tắt Timer0. - Nếu timer đang chạy (TR0 =1) thì khi nhấn nút (TR0 = 0) → tạm dừng và ngược lại. - Trong hàm reset, ta lưu lại các biến SECOND, MILISECOND về 0 để đặt lại
Hàm ngắt Timer <pre> NGAT_TIMER: INC MILISECOND CJNE MILISECOND, #100, NEXT INC SECOND MOV MILISECOND, #00H CJNE SECOND, #100, NEXT MOV SECOND, #00H NEXT: MOV TH0, #0DCH MOV TL0, #01H RETI </pre>	- Mỗi lần Timer tràn (10ms), tăng MILISECOND lên 1. - Nếu MILISECOND đủ 100 → tức là đã đủ 1 giây → tăng SECOND - Nếu đủ 100 giây → đặt lại SECOND về 0 (đồng hồ quay vòng lại từ đầu). - Nạp lại giá trị ban đầu cho Timer ở hàm NEXT.

Hàm tăng 1 giây khi nút nhấn P0.0 được nhấn <pre> NGAT_TANG1: INC SECOND CJNE SECOND, #100, INCREASE1 MOV SECOND, #99 INCREASE1: MOV MILLISECOND, #0 RET </pre>	- INC SECOND: tăng biến đếm giây lên 1. - Nếu SECOND = 100 thì đặt lại SECOND = 99 để không vượt quá 99 giây. - MOV MILLISECOND, #0: đặt lại mili giây về 0 sau khi thay đổi số giây
Hàm giảm 1 giây khi nút nhấn P0.1 được nhấn <pre> NGAT_GIAM1: CJNE SECOND, #00, DECREASE1 RET DECREASE1: DEC SECOND MOV MILLISECOND, #0 RET </pre>	- CJNE SECOND, #00, DECREASE1: nếu SECOND $\neq$ 0 thì nhảy đến DECREASE1 để trừ giây. - Nếu SECOND = 0: không làm gì cả, nếu không thì sẽ bị âm. - DEC SECOND: giảm 1 giây. - MOV MILLISECOND, #0: đặt lại mili giây về 0 sau khi thay đổi số giây

3. Exercise report.